

BytecodeVM Interpreter - Complete Technical Documentation

Project path: /home/adminn/BytecodeVM_Interpreter

Language: C++17

Documentation date: 2026-05-17

1. Executive Summary

BytecodeVM_Interpreter is a small educational compiler and virtual machine project written in C++. It implements a complete language execution pipeline: source text is tokenized by a lexer, parsed into an abstract syntax tree, compiled into bytecode instructions, and executed by a stack-based virtual machine.

The current program in `main.cpp` is hardcoded to compute factorial recursively:

```
fact(5)

func fact(n) {
    if n == 0 {
        1
    }
    else {
        n * fact(n - 1)
    }
}
```

A validation build using `g++ -std=c++17` produced:

```
Result = 120
```

2. Full Project Flow

The project follows this end-to-end architecture:

```

Source Program String
    |
    v
Lexer
    |
    v
Vector<Token>
    |
    v
Parser
    |
    v
AST Tree
    |
    v
Compiler
    |
    v
Vector<Instruction> + Function Table
    |
    v
Stack-Based Virtual Machine
    |
    v
Integer Result

```

Stage	Responsible Files	Input	Output
Entry Point	main.cpp	Hardcoded source string	Orchestrates all components
Lexing	lexer.h, lexer.cpp, token.h	Raw source code text	Tokens
Parsing	parser.h, parser.cpp, ast.h	Tokens	AST nodes
Compilation	compiler.h, compiler.cpp, instruction.h, opcode.h	AST	Bytecode instructions
Execution	vm.h, vm.cpp	Bytecode instructions and function table	Final integer result

3. Folder And File Structure

```
BytecodeVM_Interpreter/
```

```
|— ast.h
|— compiler.cpp
|— compiler.h
|— instruction.h
|— interpreter
|— lexer.cpp
|— lexer.h
|— main.cpp
|— opcode.h
|— parser.cpp
|— parser.h
|— token.h
|— vm
|— vm.cpp
└— vm.h
```

File	Purpose
<code>main.cpp</code>	Creates the source program, runs lexer, parser, compiler, and VM, then prints the result.
<code>token.h</code>	Defines token types and the <code>Token</code> class.
<code>lexer.h</code>	Declares the <code>Lexer</code> class.
<code>lexer.cpp</code>	Implements scanning of characters into tokens.
<code>ast.h</code>	Defines AST node classes for numbers, binary operations, variables, assignment, conditions, loops, functions, and calls.
<code>parser.h</code>	Declares recursive descent parser methods.
<code>parser.cpp</code>	Builds the AST from tokens.
<code>opcode.h</code>	Defines bytecode operation names.
<code>instruction.h</code>	Defines the <code>Instruction</code> class used by the VM.
<code>compiler.h</code>	Declares the <code>Compiler</code> class and its bytecode/function-table state.

`compiler.cpp` Converts AST nodes into bytecode instructions.

`vm.h` Declares `Frame` and `VM`.

`vm.cpp` Executes bytecode using a stack and call frames.

`interpreter,`
`vm` ELF executable artifacts in the repository.

4. Technology Used

Technology	How It Is Used
C++17	Main implementation language. The project uses classes, vectors, maps, strings, enums, and runtime type checks through <code>dynamic_cast</code> .
Standard Template Library	<code>std::vector</code> stores tokens, AST body lists, bytecode instructions, VM stack, and frames. <code>std::map</code> stores variables and function addresses. <code>std::string</code> stores source text and token values.
Recursive Descent Parsing	The parser has one method per grammar level: <code>expr</code> , <code>term</code> , <code>factor</code> , <code>comparison</code> , and statement parsers.
Abstract Syntax Tree	The parser creates object nodes representing the structure of the source program.
Bytecode	The compiler lowers high-level AST nodes into compact VM instructions.
Stack Virtual Machine	The VM evaluates expressions by pushing values, popping operands, and pushing results.
Call Frames	Each function call creates a frame containing local variables and a return address.

5. Language Supported By The Project

5.1 Supported Features

- Integer literals: `1`, `100`, `42`
- Variables: `x`, `total`, `fact`
- Assignment: `x = 10`
- Arithmetic: `+`, `-`, `*`, `/`
- Comparison: `>`, `<`, `==`
- Conditionals: `if ... else ...`
- Loops: `while ...`
- Function definitions: `func name(params) { ... }`
- Function calls: `name(args)`
- Recursion, as shown by `fact`

5.2 Unsupported Or Missing Features

- No strings.
- No floating-point numbers.
- No comments.
- No semicolons.
- No explicit `return` keyword.
- No user input.
- No `print` statement inside the interpreted language.
- No arrays, objects, classes, or modules.
- No unary minus, so `-5` is not directly parsed as one numeric literal.
- No `<=`, `>=`, or `!=`.
- No syntax error reporting.
- No runtime error reporting.

6. Token System

The token system is declared in `token.h`. A token is the smallest meaningful piece of source code. For example, the source `x = 5 + 2` becomes:

```
IDENTIFIER("x")
```

```
EQUAL("=")
NUMBER("5")
PLUS("+")
NUMBER("2")
EOF_TOKEN("")
```

6.1 TokenType Enum

```
enum TokenType {
    NUMBER,
    IDENTIFIER,

    PLUS,
    MINUS,
    STAR,
    SLASH,

    EQUAL,
    EQUAL_EQUAL,

    GREATER,
    LESS,

    LPAREN,
    RPAREN,

    LBRACE,
    RBRACE,

    IF,
    ELSE,
    WHILE,
    FUNC,
    COMMA,

    EOF_TOKEN
};
```

6.2 Token Class

```
class Token {
public:
    TokenType type;
    string value;
```

```

Token() {}

Token(TokenType t, string v) {
    type = t;
    value = v;
}

};

```

The `type` tells the compiler what category the token belongs to. The `value` stores the exact text, such as `"123"`, `"+"`, or `"fact"`.

7. Lexer

The lexer is declared in `lexer.h` and implemented in `lexer.cpp`. Its job is to scan the raw source string character by character and produce a vector of tokens.

7.1 Lexer State

```

class Lexer {
public:
    string text;
    int pos;
    char currentChar;

    Lexer(string input);
    void advance();
    vector<Token> tokenize();
    string identifier();
};

```

Member	Meaning
<code>text</code>	The complete source program.
<code>pos</code>	Current character index.
<code>currentChar</code>	The current character being scanned.

7.2 Lexer::advance

Moves one character forward. If the lexer reaches the end of the input, it stores `'\0'` as the end marker.

7.3 Lexer::identifier

Reads a sequence of letters, digits, or underscores. After reading the text, `tokenize` decides whether the text is a keyword such as `if` or a normal identifier such as `total`.

7.4 Lexer::tokenize

Main scanner loop. It recognizes numbers, identifiers, keywords, operators, parentheses, braces, and commas. Unknown characters are skipped by calling `advance()`.

Compiler engineering note: silently skipping unknown characters is simple, but a production compiler should report lexical errors. For example, `@` should probably produce an error instead of being ignored.

8. Abstract Syntax Tree

The AST represents the source program as structured objects. This is much easier to compile than raw tokens. All AST node classes are declared in `ast.h`.

AST Class	Represents	Important Fields
<code>NumberNode</code>	An integer literal.	<code>value</code>
<code>BinaryOpNode</code>	Arithmetic operation.	<code>left</code> , <code>op</code> , <code>right</code>
<code>VariableNode</code>	Variable read.	<code>name</code>
<code>AssignNode</code>	Variable assignment.	<code>name</code> , <code>value</code>
<code>ProgramNode</code>	Whole program.	<code>statements</code>
<code>IfNode</code>	If/else statement.	<code>condition</code> , <code>ifBody</code> , <code>elseBody</code>
<code>CompareNode</code>	Comparison expression.	<code>left</code> , <code>op</code> , <code>right</code>

WhileNode	While loop.	condition, body
FunctionDefNode	Function definition.	name, params, body
FunctionCallNode	Function call.	name, args

8.1 Example AST

Source:

```
2 + 3 * 4
```

AST shape:

```
BinaryOpNode("+")
├─ NumberNode("2")
└─ BinaryOpNode("*")
    ├─ NumberNode("3")
    └─ NumberNode("4")
```

This tree preserves precedence: multiplication is inside the right side of the addition.

9. Parser

The parser is a recursive descent parser. It consumes tokens and builds AST nodes. Its method structure encodes operator precedence.

9.1 Parser State

```
class Parser {
public:
    vector<Token> tokens;
    int pos;
    Token currentToken;

    Parser(vector<Token> t);
    void advance();
    AST* parse();
    AST* expr();
    AST* term();
```

```

    AST* factor();
    AST* statement();
    AST* program();
    AST* ifStatement();
    AST* comparison();
    AST* whileStatement();
    AST* functionDefinition();
    AST* functionCall();
};

```

9.2 Grammar Implemented

```

program      -> statement* EOF

statement    -> functionDefinition
              | ifStatement
              | whileStatement
              | assignment
              | comparison

assignment   -> IDENTIFIER "=" comparison

comparison   -> expr ((">" | "<" | "==" ) expr)*

expr         -> term (("+" | "-") term)*

term         -> factor (("*" | "/" ) factor)*

factor       -> NUMBER
              | IDENTIFIER
              | functionCall
              | "(" comparison ")"

ifStatement  -> "if" comparison "{" statement* "}"
              ("else" "{" statement* "}")?

whileStatement -> "while" comparison "{" statement* "}"

functionDefinition
              -> "func" IDENTIFIER "(" params? ")" "{" statement* "}"

functionCall  -> IDENTIFIER "(" args? ")"

```

9.3 Operator Precedence

Precedence	Parser Method	Operators
Highest	<code>factor()</code>	Numbers, variables, function calls, parentheses
Middle	<code>term()</code>	<code>*</code> , <code>/</code>
Lower	<code>expr()</code>	<code>+</code> , <code>-</code>
Lowest	<code>comparison()</code>	<code>></code> , <code><</code> , <code>==</code>

9.4 Parser Function Responsibilities

Function	Responsibility
<code>advance()</code>	Moves to the next token.
<code>parse()</code>	Entry point; calls <code>program()</code> .
<code>program()</code>	Reads statements until <code>EOF_TOKEN</code> .
<code>statement()</code>	Dispatches to function definition, if, while, assignment, or expression parsing.
<code>comparison()</code>	Parses comparison expressions.
<code>expr()</code>	Parses addition and subtraction.
<code>term()</code>	Parses multiplication and division.
<code>factor()</code>	Parses numbers, variables, function calls, and parenthesized expressions.
<code>ifStatement()</code>	Parses condition, if body, and optional else body.
<code>whileStatement()</code>	Parses loop condition and body.
<code>functionDefinition()</code>	Parses function name, parameter list, and body.
<code>functionCall()</code>	Parses function name and argument list.

Parser safety note: the parser assumes valid input. For example, it uses `tokens[pos + 1]` without a bounds check. A production parser should validate bounds and report structured syntax errors.

10. Bytecode And Instructions

Bytecode is a lower-level instruction sequence understood by the VM. Instead of directly interpreting the AST, this project compiles AST nodes into instructions such as `OP_LOAD_CONST`, `OP_ADD`, and `OP_CALL`.

10.1 Instruction Format

```
class Instruction {
public:
    OpCode opcode;
    string argument;

    Instruction(OpCode op, string arg = "") {
        opcode = op;
        argument = arg;
    }
};
```

10.2 Opcode Set

Opcode	Meaning	Stack Effect
OP_LOAD_CONST	Push integer constant.	<code>[] -> [value]</code>
OP_LOAD_NAME	Push variable value from current frame.	<code>[] -> [locals[name]]</code>
OP_STORE_NAME	Pop value into current frame variable.	<code>[value] -> []</code>
OP_ADD	Add two integers.	<code>[a, b] -> [a + b]</code>

OP_SUB	Subtract two integers.	[a, b] -> [a - b]
OP_MUL	Multiply two integers.	[a, b] -> [a * b]
OP_DIV	Integer divide two integers.	[a, b] -> [a / b]
OP_COMPARE_GT	Greater-than comparison.	[a, b] -> [a > b]
OP_COMPARE_LT	Less-than comparison.	[a, b] -> [a < b]
OP_COMPARE_EQ	Equality comparison.	[a, b] -> [a == b]
OP_JUMP_IF_FALSE LSE	Pop condition; jump if false.	[condition] -> []
OP_JUMP	Unconditional jump.	No stack change.
OP_CALL	Call a function by name.	Arguments remain on stack until function prologue stores them.
OP_RETURN	Return top stack value to caller.	[returnValue] -> caller stack receives returnValue

11. Compiler

The compiler is declared in `compiler.h` and implemented in `compiler.cpp`. It converts AST nodes into bytecode instructions.

11.1 Compiler State

```
class Compiler {
public:
    vector<Instruction> instructions;
    map<string,int> functions;

    void compile(AST* node);
};
```

Member	Meaning
--------	---------

`instructions` Generated bytecode program.

`functions` Maps function name to bytecode starting index.

11.2 Compilation Rules

AST Node	Generated Bytecode
<code>NumberNode</code>	<code>OP_LOAD_CONST</code> <code>value</code>
<code>VariableNode</code>	<code>OP_LOAD_NAME</code> <code>name</code>
<code>AssignNode</code>	Compile <code>value</code> , then <code>OP_STORE_NAME</code> <code>name</code> .
<code>BinaryOpNode</code>	Compile left, compile right, then emit arithmetic opcode.
<code>CompareNode</code>	Compile left, compile right, then emit comparison opcode.
<code>IfNode</code>	Compile condition, conditional jump, if body, jump over else, else body.
<code>WhileNode</code>	Mark loop start, compile condition, conditional exit jump, body, jump back.
<code>FunctionDefNode</code>	Register function address, store parameters, compile body, emit <code>OP_RETURN</code> .
<code>FunctionCallNode</code>	Compile each argument, then emit <code>OP_CALL</code> <code>name</code> .
<code>ProgramNode</code>	Emit initial jump, compile function definitions first, patch jump, compile top-level statements.

11.3 Why ProgramNode Emits An Initial Jump

Function definitions are compiled into the same instruction stream as executable code. If execution started at instruction zero and the first instructions were a function body, the VM would accidentally execute the function without a call. To prevent that, the compiler emits:

```
0: OP_JUMP main_start
1: function bytecode...
```

```
...
main_start:
N: top-level program bytecode...
```

This layout also lets top-level statements call functions that are defined later in source order.

12. Virtual Machine

The VM is declared in `vm.h` and implemented in `vm.cpp`. It executes the bytecode generated by the compiler.

12.1 VM State

```
class Frame {
public:
    map<string,int> locals;
    int returnAddress;
};

class VM {
public:
    vector<int> stack;
    vector<Frame> frames;
    map<string,int> functions;

    int getVariable(string name);
    void setVariable(string name, int value);
    int run(vector<Instruction>& code);
};
```

State	Purpose
stack	Evaluation stack for expression values and function return values.
frames	Call stack. Each frame stores local variables and return address.
functions	Function table copied from the compiler.

12.2 VM::run

`run` starts by creating a global frame with return address `-1`. Then it loops over bytecode using an

instruction pointer:

```
int ip = 0;

while (ip < code.size()) {
    Instruction instr = code[ip];
    switch (instr.opcode) {
        ...
    }
    ip++;
}
```

Jump and call instructions use `continue` after changing `ip`, because they do not want the normal `ip++` to run.

12.3 Call Frames

A call frame represents one active function call. For the recursive call chain `fact(5)`, the VM creates frames roughly like this:

```
Global frame
fact(n = 5)
fact(n = 4)
fact(n = 3)
fact(n = 2)
fact(n = 1)
fact(n = 0)
```

When `fact(0)` returns 1, each older frame resumes, multiplies by its own `n`, and returns.

13. Entry Point

`main.cpp` connects the whole pipeline:

```
Lexer lexer(input);
vector<Token> tokens = lexer.tokenize();

Parser parser(tokens);
AST* tree = parser.parse();

Compiler compiler;
```



```
compiler.compile(tree);

VM vm;
vm.functions = compiler.functions;

int result = vm.run(compiler.instructions);
```

The VM needs `compiler.functions` because `OP_CALL` stores only the function name. The VM uses the map to resolve the function name to an instruction index.

14. Detailed Dry Run 1: Arithmetic And Assignment

14.1 Program

```
x = 2 + 3 * 4
x + 1
```

14.2 Tokens

```
IDENTIFIER("x")
EQUAL("=")
NUMBER("2")
PLUS("+")
NUMBER("3")
STAR("*")
NUMBER("4")
IDENTIFIER("x")
PLUS("+")
NUMBER("1")
EOF_TOKEN("")
```

14.3 AST

```
ProgramNode
├─ AssignNode("x")
│   └─ BinaryOpNode("+")
│       ├── NumberNode("2")
│       └─ BinaryOpNode("*")
│           ├── NumberNode("3")
│           └─ NumberNode("4")
```

```
└─ BinaryOpNode("+")
  └─ VariableNode("x")
    └─ NumberNode("1")
```

14.4 Bytecode

```
0: OP_JUMP 1
1: OP_LOAD_CONST 2
2: OP_LOAD_CONST 3
3: OP_LOAD_CONST 4
4: OP_MUL
5: OP_ADD
6: OP_STORE_NAME x
7: OP_LOAD_NAME x
8: OP_LOAD_CONST 1
9: OP_ADD
```

14.5 VM Dry Run

IP	Instruction	Stack Before	Stack After	Locals
0	OP_JUMP 1	[]	[]	{}
1	OP_LOAD_CONST 2	[]	[2]	{}
2	OP_LOAD_CONST 3	[2]	[2, 3]	{}
3	OP_LOAD_CONST 4	[2, 3]	[2, 3, 4]	{}
4	OP_MUL	[2, 3, 4]	[2, 12]	{}
5	OP_ADD	[2, 12]	[14]	{}
6	OP_STORE_NAME x	[14]	[]	{x: 14}
7	OP_LOAD_NAME x	[]	[14]	{x: 14}
8	OP_LOAD_CONST 1	[14]	[14, 1]	{x: 14}
9	OP_ADD	[14, 1]	[15]	{x: 14}

Final result: 15.

15. Detailed Dry Run 2: If/Else

15.1 Program

```
x = 7
if x > 5 {
    x * 2
}
else {
    x - 2
}
```

15.2 Concept

The condition $x > 5$ is compiled first. The VM pushes 1 for true or 0 for false. `OP_JUMP_IF_FALSE` consumes that value. If it is false, execution jumps to the else block.

15.3 Approximate Bytecode

```
0: OP_JUMP 1
1: OP_LOAD_CONST 7
2: OP_STORE_NAME x
3: OP_LOAD_NAME x
4: OP_LOAD_CONST 5
5: OP_COMPARE_GT
6: OP_JUMP_IF_FALSE 11
7: OP_LOAD_NAME x
8: OP_LOAD_CONST 2
9: OP_MUL
10: OP_JUMP 14
11: OP_LOAD_NAME x
12: OP_LOAD_CONST 2
13: OP_SUB
```

15.4 Dry Run

Step	Action	Result
1	Store $x = 7$	Locals become $\{x: 7\}$.
2	Load x and 5	Stack becomes $[7, 5]$.
3	Run <code>OP_COMPARE_GT</code>	$7 > 5$ is true, stack becomes $[1]$.
4	Run <code>OP_JUMP_IF_FALSE</code> 11	Condition is true, so no jump.
5	Run if body $x * 2$	Stack becomes $[14]$.
6	Run <code>OP_JUMP</code> 14	Skip else body.

Final result: 14.

16. Detailed Dry Run 3: While Loop

16.1 Program

```
i = 0
sum = 0
while i < 5 {
    sum = sum + i
    i = i + 1
}
sum
```

16.2 Expected Result

This computes:

```
0 + 1 + 2 + 3 + 4 = 10
```

16.3 Loop State Dry Run

Iteration	Before Condition	Condition	After Body
-----------	------------------	-----------	------------

```
1      i = 0, sum = 0  0 < 5 true  sum = 0, i = 1
2      i = 1, sum = 0  1 < 5 true  sum = 1, i = 2
3      i = 2, sum = 1  2 < 5 true  sum = 3, i = 3
4      i = 3, sum = 3  3 < 5 true  sum = 6, i = 4
5      i = 4, sum = 6  4 < 5 true  sum = 10, i = 5

Exit   i = 5, sum = 10 5 < 5 false Jump out of loop.
```

16.4 VM Concept

A while loop is implemented with two jumps:

```
loop_start:
  condition
  OP_JUMP_IF_FALSE loop_end
  body
  OP_JUMP loop_start
loop_end:
```

Final result: 10.

17. Detailed Dry Run 4: Recursive Factorial

17.1 Program

```
fact(5)

func fact(n) {
  if n == 0 {
    1
  }
  else {
    n * fact(n - 1)
  }
}
```

```
}
```

17.2 High-Level Meaning

```
fact(5)
= 5 * fact(4)
= 5 * 4 * fact(3)
= 5 * 4 * 3 * fact(2)
= 5 * 4 * 3 * 2 * fact(1)
= 5 * 4 * 3 * 2 * 1 * fact(0)
= 5 * 4 * 3 * 2 * 1 * 1
= 120
```

17.3 Approximate Bytecode Shape

```
0:  OP_JUMP main_start

fact_start:
1:  OP_STORE_NAME n
2:  OP_LOAD_NAME n
3:  OP_LOAD_CONST 0
4:  OP_COMPARE_EQ
5:  OP_JUMP_IF_FALSE else_start
6:  OP_LOAD_CONST 1
7:  OP_JUMP if_end

else_start:
8:  OP_LOAD_NAME n
9:  OP_LOAD_NAME n
10: OP_LOAD_CONST 1
11: OP_SUB
12: OP_CALL fact
13: OP_MUL

if_end:
14: OP_RETURN

main_start:
15: OP_LOAD_CONST 5
16: OP_CALL fact
```

The exact numeric addresses depend on the compiler's generated instruction count, but this is the actual logical layout.

17.4 Function Table

```
functions["fact"] = fact_start
```

17.5 Call Stack Dry Run

Call	Frame Local n	Condition	Action
fact(5)	5	5 == 0 false	Compute 5 * fact(4).
fact(4)	4	4 == 0 false	Compute 4 * fact(3).
fact(3)	3	3 == 0 false	Compute 3 * fact(2).
fact(2)	2	2 == 0 false	Compute 2 * fact(1).
fact(1)	1	1 == 0 false	Compute 1 * fact(0).
fact(0)	0	0 == 0 true	Return 1.

17.6 Return Unwinding

Returning To	Operation	Return Value
fact(1)	1 * 1	1
fact(2)	2 * 1	2
fact(3)	3 * 2	6
fact(4)	4 * 6	24
fact(5)	5 * 24	120

18. Function Calls And Important Parameter Limitation

Function arguments are compiled left to right:

```
func sub(a, b) {  
    a - b  
}  
  
sub(10, 3)
```

The call compiles arguments as:

```
OP_LOAD_CONST 10  
OP_LOAD_CONST 3  
OP_CALL sub
```

The stack before entering `sub` is:

```
[10, 3]
```

The function prologue stores parameters using `OP_STORE_NAME` in parameter order:

```
OP_STORE_NAME a  
OP_STORE_NAME b
```

Since `OP_STORE_NAME` pops from the top of the stack, this binds:

```
a = 3  
b = 10
```

Therefore `sub(10, 3)` currently behaves like `3 - 10`, returning `-7`.

Known bug: multi-argument function calls bind parameters in reverse order. A fix would be to store parameters in reverse order or compile arguments right to left.

19. Compiler And VM Concepts You Should Know

19.1 Lexer

A lexer converts characters into tokens. This simplifies parsing because the parser does not need to reason about individual characters. It sees meaningful categories such as `NUMBER`, `IDENTIFIER`, and `PLUS`.

19.2 Parser

A parser converts tokens into a structured representation. This project uses recursive descent, meaning grammar rules are implemented as C++ functions that call each other.

19.3 AST

An AST removes unnecessary syntax details and keeps the semantic structure. For example, parentheses influence the tree shape but are not usually stored as AST nodes.

19.4 Compiler

A compiler translates a high-level representation into a lower-level representation. Here, it translates AST into bytecode. In larger systems, a compiler might also perform semantic analysis, type checking, optimization, and machine-code generation.

19.5 Bytecode

Bytecode is an instruction format for a virtual machine. It is usually simpler than source code and more portable than native machine code. Examples include Java bytecode, Python bytecode, Lua bytecode, and WebAssembly-like instruction streams.

19.6 Stack Machine

A stack machine uses an implicit operand stack. Arithmetic instructions do not name registers. Instead, they pop operands from the stack and push results back.

```
OP_LOAD_CONST 2    stack: [2]
OP_LOAD_CONST 3    stack: [2, 3]
OP_ADD             stack: [5]
```

19.7 Instruction Pointer

The instruction pointer, often called `ip` or program counter, stores the index of the next instruction to execute. Normal instructions increment it by one. Jumps and calls modify it directly.

19.8 Call Stack

The call stack stores active function calls. Each frame contains local variables and where to resume after the

function returns.

19.9 Return Address

A return address is the instruction index after the call instruction. When a function returns, the VM sets `ip` back to this address.

20. Important Design Observations

20.1 Expression Statements Leave Values On Stack

If a statement is an expression, it leaves its value on the stack. This is why the last expression can become the program result. Assignment statements do not leave a value because `OP_STORE_NAME` pops the assigned value.

20.2 Undefined Variables Become Zero

`getVariable` uses:

```
return frames.back().locals[name];
```

In `std::map`, using `operator[]` creates a missing key with a default value. For `int`, that default is `0`. So reading an undefined variable silently gives `0`.

20.3 Function Definitions Are Not First-Class Values

Functions are stored in the compiler and VM function table. They are not values that can be assigned to variables, passed as arguments, or returned from other functions.

20.4 No Lexical Closures

A function only accesses its current frame locals. It does not capture variables from the enclosing scope.

20.5 Runtime Type System Is Integer-Only

The VM stack is `vector<int>`. Comparisons push integer `0` or `1`.

21. Error Handling Gaps

Area	Current Behavior	Better Behavior
Invalid character	Skipped by lexer.	Report lexical error with line and column.
Missing brace	Parser can loop or read invalid tokens.	Report expected <code>}</code> .
Undefined variable	Returns <code>0</code> .	Runtime error: undefined variable.
Division by zero	Native crash or undefined behavior risk.	Runtime error: division by zero.
Wrong function arity	No validation.	Runtime or compile-time arity error.
Unknown function	<code>functions[name]</code> can default to <code>0</code> .	Runtime error: unknown function.

22. Suggested Test Cases

22.1 Arithmetic Precedence

```
2 + 3 * 4
```

Expected result: 14.

22.2 Parentheses

```
(2 + 3) * 4
```

Expected result: 20.

22.3 Assignment

```
x = 10
y = x + 5
y * 2
```

Expected result: 30.

22.4 If True

```
x = 9
if x > 5 {
    100
}
else {
    200
}
```

Expected result: 100.

22.5 If False

```
x = 3
if x > 5 {
    100
}
else {
    200
}
```

Expected result: 200.

22.6 While Sum

```
i = 0
sum = 0
while i < 5 {
    sum = sum + i
    i = i + 1
}
sum
```

Expected result: 10.

22.7 Recursive Factorial

```
fact(5)

func fact(n) {
```

```

    if n == 0 {
        1
    }
    else {
        n * fact(n - 1)
    }
}

```

Expected result: 120.

22.8 Multi-Argument Bug Demonstration

```

sub(10, 3)

func sub(a, b) {
    a - b
}

```

Expected mathematically: 7. Current project behavior: likely -7, because parameters bind in reverse order.

23. How To Extend The Project

23.1 Add A Return Keyword

Required changes:

- Add RETURN token type.
- Make lexer recognize `return`.
- Add `ReturnNode` to `ast.h`.
- Parse return statements in `Parser::statement`.
- Compile `ReturnNode` to expression bytecode plus `OP_RETURN`.

23.2 Add Unary Minus

Modify `factor()` so if it sees `MINUS`, it parses the next factor and creates either:

```

0 - expression

```

or a dedicated UnaryOpNode.

23.3 Add Error Reporting

Add line and column fields to tokens, then use parser helper functions like:

```
expect(TokenType expected, string message)
```

This would let the parser say things like: Expected ')' after function arguments at line 3, column 14.

23.4 Fix Multi-Argument Function Calls

Option one: store function parameters in reverse order inside

Compiler::compile(FunctionDefNode*). Option two: compile call arguments right to left.

Option three: change OP_CALL to create a frame and explicitly move arguments into locals using known arity metadata.

24. Current Build Instructions

Build without overwriting the tracked executable:

```
cd /home/adminn/BytecodeVM_Interpreter
g++ -std=c++17 main.cpp lexer.cpp parser.cpp compiler.cpp vm.cpp -o
/tmp/bytecodevm_interpreter
/tmp/bytecodevm_interpreter
```

Build into the project executable:

```
cd /home/adminn/BytecodeVM_Interpreter
g++ -std=c++17 main.cpp lexer.cpp parser.cpp compiler.cpp vm.cpp -o interpreter
./interpreter
```

25. Final Mental Model

To understand this project as a compiler engineer, keep this mental model:

Lexer asks:

"What are the words and symbols?"

Parser asks:

"What structure do those words and symbols form?"

AST stores:

"This is the program's tree-shaped meaning."

Compiler asks:

"What VM instructions implement this tree?"

VM asks:

"What should each instruction do to the stack, frames, variables, and instruction pointer?"

The project is small, but it contains the essential architecture used by many real interpreters: tokenization, parsing, AST construction, bytecode generation, stack execution, control flow, function calls, recursion, and call frames.